**UNIVERSITY OF EASTERN PANGASINAN**

College of Information Technology

CCM101 - Cloud Computing

# ENTERPRISE CLOUD ARCHITECT OPERATIONAL MANUAL

## Secure Multi-Tier Cloud Website - WordPress + MariaDB


**Prepared by:**

Cabanada, Paul Arwind C.

Distor, Lorene Joy R.

Lavarias, Nathan Joshua D.

Peralta, Jane Princess L.


BSIT 4G

Jenkielyn C. Torres

October 2026

# TABLE OF CONTENTS

**1. Introduction**
1.1 Purpose of the Manual ..... 3
1.2 Project Overview ..... 3
1.3 Objectives ..... 3
1.4 System Components ..... 3
1.5 System Architecture ..... 3
**2. Prepare the Environment** ..... 4
2.1 Create the Ubuntu Server VM ..... 4
2.2 Install Docker ..... 4
**3. Deploy the Application** ..... 5
3.1 Create the Project Directory ..... 5
3.2 Configure Passwords ..... 5
3.3 Start and Verify Services ..... 5
3.4 Open the Website ..... 5
**4. Testing and Persistent Storage** ..... 6
4.1 Test the Website ..... 6
4.2 Verify the Containers and Volumes ..... 6
4.3 Test Checklist ..... 6
4.4 Test Data Persistence After VM Reboot ..... 6
**5. Security Configuration** ..... 7
5.1 Review Network and Firewall ..... 7
5.2 Protect the Database and Secrets ..... 8
**6. Backup and Automation** ..... 8
6.1 Install the Backup Script ..... 8
6.2 Schedule the Backup with Cron ..... 8
6.3 Back Up WordPress Media File ..... 9
6.4 Recovery Notes ..... 9
**7. Maintenance and Troubleshooting** ..... 9
7.1 Routine Commands ..... 9
7.2 Common Problems ..... 10
**8. GitHub Repository** ..... 11
**9. Final Reflection** ..... 11

# 1. INTRODUCTION

## 1.1 Purpose of the Manual

This manual explains how to prepare, deploy, test, secure, and maintain a small multi-tier cloud website. Follow the steps in order and record the actual results from your own machine.

## 1.2 Project Overview

The project uses Ubuntu Server inside Oracle VirtualBox. Docker Compose runs WordPress as the web application and MariaDB as its database. The services communicate on a private Docker network. Named volumes keep website and database files between container restarts.

## 1.3 Objectives

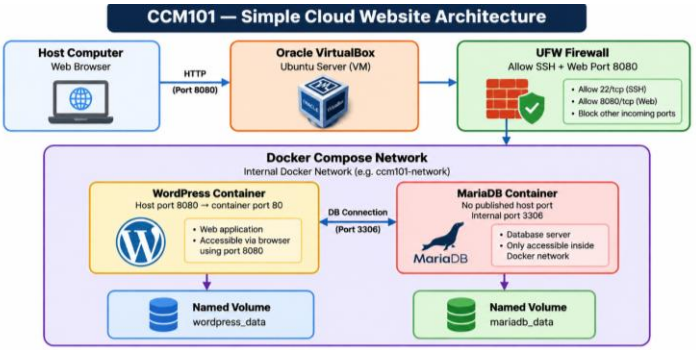
- Create a virtualized Ubuntu Server environment.
- Deploy two linked containers using Docker Compose.
- Use named volumes for persistent storage.
- Restrict incoming connections with UFW.
- Create a Bash database backup and schedule it with Cron.
- Document tests, maintenance steps, and troubleshooting.

## 1.4 System Components

- Host computer and browser — used to manage and access the project.
- Oracle VirtualBox — runs the Ubuntu Server virtual machine.
- Ubuntu Server — host operating system for Docker.
- Docker Engine and Compose — run and manage the containers.
- WordPress — provides the website interface.
- MariaDB — stores WordPress data.
- UFW — controls host firewall rules.
- Bash and Cron — automate database backups.

## 1.5 System Architecture

Browser → VirtualBox/Ubuntu Server → Docker Compose network → WordPress container → MariaDB container. WordPress and MariaDB use named volumes. Only the website port is published to the host; MariaDB port 3306 remains internal.




# 2. PREPARE THE ENVIRONMENT

## 2.1 Create the Ubuntu Server VM.

1. Download an Ubuntu Server LTS ISO from the official Ubuntu website.
2. Open VirtualBox and create a virtual machine named CCM101-Ubuntu. Select Linux/Ubuntu (64-bit).
3. Assign at least 2 CPU cores, 2–4 GB RAM if available, and 20 GB or more virtual disk space.
4. Attach the ISO and install Ubuntu Server. Create a username and a strong password.
5. Configure networking. Bridged networking is convenient on a trusted classroom LAN; NAT requires port forwarding. Follow the instructor's network rules.
6. Log in and update packages.

```
sudo apt update && sudo apt upgrade -y
hostname -I
lsb_release -a
```

## 2.2 Install Docker

Use Docker's official Ubuntu installation instructions because package names can change. Install Docker Engine and the Docker Compose plugin, then verify:

```
docker --version
docker compose version
sudo systemctl enable --now docker
sudo systemctl status docker
```

If you use Docker commands without sudo, follow Docker's official guidance sudo mkdir -p /opt/ccm-cloud-siteprivileges.

# 3. DEPLOY THE APPLICATION

## 3.1 Create the Project Directory

```
sudo mkdir -p /opt/ccm-cloud-site
sudo chown "$USER":"$USER" /opt/ccm-cloud-site
cd /opt/ccm-cloud-site
```

Copy docker-compose.yml, .env.example, and backup.sh from the project files into this directory. Confirm with ls -la.

## 3.2 Configure Passwords

7. Create a private environment file from the template.
8. Edit the file and replace both sample values with different, long passwords.
9. Restrict access to the file and do not upload it to GitHub.

```
cp .env.example .env
nano .env
chmod 600 .env
```

The file must contain WORDPRESS_DB_PASSWORD and MARIADB_ROOT_PASSWORD. Keep the values private. The database password used by WordPress must match the password used by MariaDB.

## 3.3 Start and Verify Services

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs --tail 50
```

The configuration check should finish without an error. Wait for both services to start. If one is restarting or unhealthy, inspect its logs before continuing.

## 3.4 Open the Website

From a browser on a machine that can reach the VM, open http://SERVER-IP:8080 and replace SERVER-IP with the address shown by hostname -I. If

using VirtualBox NAT, configure a host-to-guest port-forwarding rule for port 8080. Complete the WordPress setup page.

If the browser cannot connect, check the VM IP, VirtualBox network mode, Docker port mapping, and firewall rules.

# 4. TESTING AND PERSISTENT STORAGE

## 4.1 Test the Website

10. Finish the WordPress initial setup.
11. Sign in to the WordPress dashboard.
12. Create a temporary test post with a clear title and save it.
13. Open the post in the browser and confirm that it appears.

## 4.2 Verify the Containers and Volumes

```
docker compose ps
docker volume ls
docker compose restart
docker compose ps
```

After the containers restart, reload the site and check that the test post still exists. The named volumes wordpress_data and mariadb_data are defined in the Compose file.

## 4.3 Test Checklist

- Compose configuration validates without errors.
- WordPress and MariaDB containers are running.
- Website opens from the host browser.
- Test post can be saved and viewed.
- Test post remains after a container restart.
- Database port 3306 is not published to the host.
- Backup script creates a non-empty SQL file.
- UFW status and scheduled Cron entry are checked.

Only mark a test as passed after you personally run it. Add screenshots of real results to your submission.

## 4.4 Test Data Persistence After VM Reboot

This test checks whether the website and saved content remain available after the Ubuntu Server virtual machine is rebooted. Perform the test after

confirming that the application is working and the named volumes are present.

1. Save a temporary test post in WordPress and confirm that it opens in the browser.

2. Shut down Ubuntu Server safely, then start the existing CCM101-Ubuntu virtual machine again.

3. Log in and run the following commands:

```
cd /opt/ccm-cloud-site
sudo docker compose ps
sudo docker volume ls
```

4. If the containers are stopped, start the existing services:

```
sudo docker compose start
```

5. Wait for the services to run, open the website, and confirm that the test post still exists.

# 5. SECURITY CONFIGURATION

## 5.1 Review Network and Firewall

```
sudo ufw status verbose
```

Before enabling a firewall, confirm how you connect to the VM. If you administer the VM over SSH, allow SSH first to avoid locking yourself out. The following is an example for a lab VM that needs SSH and web access on port 8080:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 8080/tcp
sudo ufw enable
sudo ufw status numbered
```

If SSH is not used, do not open port 22 unnecessarily. Restrict access to trusted networks when practical. Docker-published ports can interact with

UFW rules differently from normal host services, so test access from another machine and follow your instructor's requirements.

### 5.2 Protect the Database and Secrets

- Do not publish MariaDB port 3306 to the host.
- Keep .env private and exclude it from Git.
- Use strong, different database passwords.
- Do not commit SQL backups, passwords, or private screenshots.
- Keep Ubuntu and container images updated after reviewing changes.

## 6. BACKUP AND AUTOMATION

### 6.1 Install the Backup Script

Copy backup.sh into /opt/ccm-cloud-site and confirm that the .env file exists. The script uses the MariaDB root password from the Compose environment, creates a timestamped SQL dump, checks that it is non-empty, and removes SQL backups older than seven days.

```
cd /opt/ccm-cloud-site
chmod +x backup.sh
./backup.sh
ls -lh backups
```

A successful run should print the path of the backup file and create a non-empty .sql file. If it fails, inspect the error and container logs. Do not claim the backup succeeded until the file is verified.

### 6.2 Schedule the Backup with Cron

```
crontab -e
```

Add the following line to run the backup every day at 2:00 AM:

```
0 2 * * * /bin/bash /opt/ccm-cloud-site/backup.sh >> /opt/ccm-cloud-site/backups/cron.log 2>&1
crontab -l
```

Cron uses the VM's local time zone. Verify that the entry is listed, then check the backup directory and cron.log after the scheduled run. The schedule alone does not prove a backup completed.

### 6.3 Back Up WordPress Media Files

The SQL backup protects database content, but WordPress uploads and other files may be stored in the WordPress named volume. Back up that volume as part of a fuller recovery plan.

First, confirm the actual volume name:

```
sudo docker volume ls
```

Create a compressed archive of the WordPress volume. Verify the exact volume name on your VM before running the example below:

```
cd /opt/ccm-cloud-site
sudo docker run --rm -v ccm-cloud-site_wordpress_data:/data:ro -v "$PWD/backups":/backup alpine tar -czf /backup/wordpress-volume-backup.tar.gz -C /data .
```

Verify that the archive exists:

```
sudo ls -lh /opt/ccm-cloud-site/backups/
```

Keep backup files private and do not upload them to a public repository.

### 6.4 Recovery Notes

The SQL dump contains database content, but it does not include all WordPress media files. For a fuller recovery plan, also back up the WordPress volume and document a restore procedure. Test restoration in a safe environment before relying on a backup.

Test restoration only in a safe test environment, not on the live project. Confirm that the SQL backup and WordPress volume archive exist before starting.

## 7. MAINTENANCE AND TROUBLESHOOTING

### 7.1 Routine Commands

```
cd /opt/ccm-cloud-site
docker compose ps
docker ps
docker compose logs --tail 50
docker compose restart
docker compose stop
```

| |
|---|
| **docker compose start** |
| **docker volume ls** |
| **sudo ufw status verbose** |

### 7.2 Common Problems

Website does not load:

- Confirm the VM is running and use the correct IP address.
- Run docker compose ps and inspect logs.
- Check port 8080 mapping, VirtualBox networking, and firewall access.

WordPress cannot connect to MariaDB:

- Confirm both containers are running.
- Check the .env values and service hostname db:3306.
- Inspect logs using docker compose logs --tail 50.

Compose cannot find its file:

- Run pwd and ls -la.
- Change to /opt/ccm-cloud-site before running Compose commands.

Backup fails:

- Check that MariaDB is running and .env contains the required password.
- Check directory permissions and script output.
- Verify a non-empty SQL file was created.

Data seems missing:

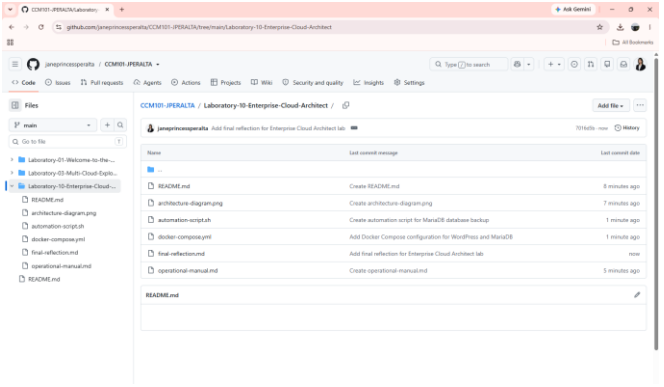
- Confirm the correct Compose project and named volumes are in use.
- Do not run docker compose down -v; it can delete persistent data.
- Inspect logs and volume configuration before making changes.

## 8. GITHUB REPOSITORY

The Mission 10 folder should contain the following files:

**Laboratory-10-Enterprise-Cloud-Architect/**
- **README.md**
- **architecture-diagram.png**
- **backup.sh**
- **docker-compose.yml**
- **final-reflection.md**
- **operational-manual.md**

Create or open the course GitHub repository and add a folder named Laboratory-10-Enterprise-Cloud-Architect. Include:



## 9. FINAL REFLECTION

To conclude, Through this project, we learned how to install and use Ubuntu Server in VirtualBox, configure Docker Compose, and run WordPress and MariaDB containers. We also learned how to connect WordPress to MariaDB and check if both containers are running properly.

We gained hands-on experience using Docker volumes to store website and database data, checking network and firewall settings, and creating database backups using a Bash script. We also learned how to schedule automatic backups using Cron.

Overall, this project helped us understand how to deploy, test, and maintain a WordPress website using Docker Compose. It also improved our practical skills and knowledge in Cloud Computing.